HEBCPF

Holomorphic-Embedding-Based Continuation Power-Flow All-Solutions Solver

Finding all *real-valued solutions of the AC power-flow equations via the ellipsoidal formulation, holomorphic embedding, Padé approximation, and numerical continuation*

Release: MEX v4.20260715

Windows 64-bit (compiled MEX) — performance-optimized, robustness-hardened build

Method:	Holomorphic Embedding Based Continuation (HEBC)
Originating work:	D. Wu and B. Wang, *IEEE Access*, 2019
Foundations:	Elliptical branch tracing — B. Lesieutre & D. Wu, Allerton 2015; Algebraic set preserving mappings — D. Wu, Ph.D. dissertation, UW–Madison, 2017
Document:	User Guide and Reference Manual

When you publish results obtained with this solver, please cite the HEBC paper (see Section 14).

July 15, 2026

Contents

1 Overview

HEBCPF computes **all** real-valued solutions of the AC power-flow (load-flow) equations for a given network, rather than the single operating point returned by a conventional solver. Knowing the complete solution set is essential for transient-stability energy-function methods (locating type-1 unstable equilibria), voltage-stability analysis, and the study of multiple high-voltage operating points.

The solver implements the **Holomorphic Embedding Based Continuation (HEBC)** method of Wu and Wang (see Section 14). The power-flow equations are first recast in an *elliptical form* (Lesieutre and Wu, 2015) so that every solution-tracing curve is a bounded loop; the loops are then followed by a hybrid numerical scheme:

- **Holomorphic embedding** traverses the smooth (non-singular) portions of a curve in a few very large steps, using power-series / Padé evaluation of the holomorphically-embedded voltages;
- a classical **predictor–corrector continuation** takes over only in the neighbourhood of singular points (turning points / folds), where it steps carefully through the fold and hands control back to the holomorphic routine.

Each time the tracing (homotopy) parameter crosses zero, a power-flow solution has been found. Starting from one known operating point (from MATPOWER's `runpf`), the solver recursively traces the curves emanating from every discovered solution until no new solution appears.

This release adds a **consensus Padé-pole estimator** to the classic method (Section 5.5), which stabilises the placement of the holomorphic/continuation hand-off by agreeing a singularity estimate across several buses rather than trusting a single bus.

Three execution frameworks share the identical numerical hot path:

Framework	Entry script	Best for
Serial	`run_batch.m`	small cases, debugging, reproducibility
Parallel (parfor)	`run_batch_par.m`	medium cases, one worker per equation
Parallel (parfeval)	`run_batch_parfeval.m`	**large cases (recommended)**

1.1 What is new in v4

V4 retains the same equations, MEX binaries, case format, and user-facing workflows as the prior MEX solver. Its changes target the cost of repeated tracing work:

- **Cached sparse LU ordering.** The structural column ordering of the holomorphic matrix is reused within a case, avoiding repeated sparse-analysis work.
- **Sparse holomorphic assembly.** Constant network blocks and stacked quadratic-form operators are reused, while diagonal blocks are built directly as sparse matrices.
- **Allocation-free Padé evaluation.** Voltage numerator and denominator polynomials are evaluated with matrix-vector products rather than replicated power matrices.
- **Global `parfeval` work queue.** The asynchronous driver keeps a pending queue per equation and submits new work as workers finish, reducing row-barrier idle time without allowing concurrent traces of the same equation.

2 Requirements and dependencies

MATLAB (required).
Developed and tested on Windows 64-bit, MATLAB R2022a and later. Uses `sparse`, `linsolve`, `lu`, and standard linear algebra.

MATPOWER (required).
MATPOWER must be installed and on the MATLAB path. The setup stage calls MATPOWER functions including `runpf` for the initial operating point, `makeYbus` for the bus-admittance matrix, and `idx_bus`/`idx_brch` for MATPOWER matrix indices. The solver reads the bundled MATPOWER-format `caseXXX.m` data files, but does *not* ship `runpf`, `makeYbus`, `idx_bus`, `idx_brch`, or their dependencies — those come from MATPOWER (https://matpower.org).

Parallel Computing Toolbox (optional).
Required *only* for the two parallel frameworks (`run_batch_par.m`, `run_batch_parfeval.m`). The serial framework runs without it.

Compiled MEX (shipped).
Two hot-path functions are provided as Windows 64-bit binaries, `Pade_Apprxmt_mex.mexw64` and `holomorphic_cont_tri_mex.mexw64`. These `.mexw64` files are only expected to run on Windows 64-bit MATLAB. They accelerate the shipped benchmark package on supported Windows systems. The release queue-`parfeval` benchmark through 57 buses took 1.77× less aggregate wall time than pure MATLAB; see Section 8 for scope and limits. On Linux/macOS, or after editing hot-path source, you must rebuild them (Section 11) with MATLAB Coder and a configured C compiler.

KLU acceleration (shipped).
The bordered Newton solve is accelerated by the `klurf` MEX (`klurf.mexw64`, built from SuiteSparse/KLU), which is shipped and enabled automatically; the solver falls back to the dense solve if it is absent (Section 6.6). SuiteSparse/KLU is third-party software under the LGPL-2.1+ license.

Installing and compiling SuiteSparse/KLU. The `klurf` MEX links KLU from **SuiteSparse** (SPDX `LGPL-2.1+`). A Windows 64-bit `klurf.mexw64` is already bundled; rebuild only for another platform or after editing the source. You do *not* need a system-wide SuiteSparse install or its CMake build — `klurf_make.m` compiles just the handful of KLU / AMD / COLAMD / BTF sources it needs, so a working MATLAB `mex` compiler is the only prerequisite.

1. Get the SuiteSparse sources (any recent release), e.g.

   ```
   git clone --depth 1 https://github.com/DrTimothyAldenDavis/SuiteSparse
   ```

2. Copy `klurf/klurf_mex.c` and `klurf/klurf_make.m` (from the top-level `klurf/` folder) into `SuiteSparse/KLU/MATLAB/`.

3. In MATLAB, select a C compiler once and run the build from that directory:

   ```
   >> mex -setup C          % pick MSVC / gcc / clang (once per machine)
   >> cd '<...>/SuiteSparse/KLU/MATLAB'
   >> klurf_make            % compiles klurf.<mexext>, real-valued, CHOLMOD-free
   ```

4. Copy the resulting `klurf.<mexext>` (`klurf.mexw64` on Windows, `klurf.mexa64` on Linux, `klurf.mexmaci64`/`klurf.mexmaca64` on macOS) into this release folder, or anywhere on the MATLAB path. The solver detects it and switches on the refactor path automatically.

Further platform notes are in the top-level `klurf/README_KLURF.md`.

3 Installation

1. Install MATLAB and MATPOWER; confirm `runpf` works (`which runpf` returns a path).
2. Unpack this folder (`HEBCPF_MEX_v4_20260715`) anywhere on disk.
3. Add the folder to the MATLAB path each session:

```
>> cd '<path>\HEBCPF_MEX_v4_20260715'
>> addpath(pwd)
```

4. Add only one HEBCPF solver folder to the MATLAB path at a time. Do not mix this folder with the pure-MATLAB or older-release folders because they contain many functions and cases with the same names.
5. (Windows x64 only) nothing else is needed — the shipped `.mexw64` binaries are picked up automatically. On other platforms, run `build_mex` (Section 11) or switch to a pure-MATLAB build.

4 Quick start

Before running any driver, confirm MATLAB can find MATPOWER:

```
>> which runpf
>> which makeYbus
```

Both commands should return paths inside the MATPOWER installation.

From a system shell (non-interactive `-batch`):

```
:: SERIAL
matlab -sd "<path>\HEBCPF_MEX_v4_20260715" -batch "addpath(pwd); run('run_batch.m')
    "

:: PARALLEL (parfor)
matlab -sd "<path>\HEBCPF_MEX_v4_20260715" -batch "addpath(pwd); run('run_batch_par
    .m')"

:: PARALLEL (parfeval, recommended for big cases)
matlab -sd "<path>\HEBCPF_MEX_v4_20260715" -batch "addpath(pwd); run('
    run_batch_parfeval.m')"
```

From an interactive MATLAB session:

```
>> addpath(pwd)
>> run_batch            % serial batch
>> run_batch_par        % parallel (parfor)
>> run_batch_parfeval   % parallel (parfeval)
>> r = run_merged_case('case14mod','parfeval');  % one case, any mode
>> r.solutions, r.max_residual, r.wall_time
```

The two parallel drivers start a local `parpool` automatically (default worker count) if none is running. The one-time pool start-up is excluded from the reported timing.

5 User options

5.1 Where the settings live: `solver_params.m`

Every tunable constant is defined in one place — the function `solver_params.m`, which returns a `param` struct consumed by the whole solver (`hybrid_traceloops` → holomorphic step, continuation, corrector). To change any behaviour, edit the relevant line in `solver_params.m`; all three frameworks *and* the parallel workers read the same file, so one edit applies everywhere. Every field is documented inline, and Section 7 tables the ones you are most likely to touch. (The only tunable kept outside this file is the Padé `degree`, which is baked into the MEX binaries — see Section 7.)

5.2 Choosing which cases to run

Each driver has a list near the top:

```
cases_to_run = { 'case9', 'case14mod', 'case39', ... };
```

Comment out or add case names. Bundled test systems (expected solution counts, collapsing antipodal pairs, in parentheses):

`case3` (6)	`case4gs` (6)	`case4BB0` (14)	`case4BBc` (12)
`case5loop` (10)	`case5Salam` (10)	`case6ww` (6)	`case7Salam` (4)
`case9` (8)	`case9Q` (8)	`case14mod` (30)	`case33bw` (16)
`case39` (176)	`case30/case_ieee30` (472)	`case57mod` (606)	`case57` (1322)

5.3 Load scaling

Each driver contains

```
factor = 1 + 0.19*(0);   % default = 1 (nominal load)
```

Changing the multiplier (the "`(0)`") scales all bus loads and generator outputs, letting you sweep load level. Leave it at 1 to reproduce the nominal solution counts.

5.4 Choosing a framework: serial, `parfor`, `parfeval`

All three frameworks compute the *identical* solution set; they differ only in how the per-equation curve traces are distributed across processes.

Serial (`run_batch.m`)
one process traces every curve in turn. Needs no toolbox; fully deterministic and the easiest to debug.

`parfor` (`run_batch_par.m`)
MATLAB's parallel *for*-loop. The equations of one starting solution are grouped into a *batch* and dealt out to the worker pool, but the loop then **blocks until the whole batch has finished** before it proceeds. It is simple and effective, yet a batch runs only as fast as its *slowest* equation: if one curve is long or stalls, the other workers sit idle at that synchronisation barrier until it completes.

`parfeval` (`run_batch_parfeval.m`)
asynchronous task submission. Each equation is submitted individually via `parfeval`, and

results are collected in the order they *complete*, whatever that order is. A sliding window keeps the pool saturated: the instant a worker finishes a curve it is handed the next one, *without* waiting for the others. This removes the per-batch barrier and keeps every worker busy, so it is the fastest framework on large systems with uneven curve lengths (e.g. `case30`, `case57mod`). The only trade-off is that the *order* in which curves are traced is non-deterministic (it depends on which worker finishes first); the final solution set is identical, but which curve happens to discover a given solution can differ between runs.

In one line: `parfor` is *synchronous, batch-at-a-time* and blocks on the slowest curve; `parfeval` is *asynchronous, task-at-a-time* with no idle waiting — prefer it for large cases. Both start a local pool automatically and require the Parallel Computing Toolbox; the one-time pool start-up is excluded from the reported timing.

5.5 Padé pole estimation (consensus)

The hand-off from the holomorphic routine to the continuation routine is driven by an estimate of the nearest singularity, taken from the real roots (poles) of the Padé denominator. This build estimates that pole by **consensus** across a fixed set of buses (`param.pole_buses`, default `2:6`, auto-filtered to $\leq$ bus count): the true singularity sits at nearly the same magnitude on every bus, whereas spurious (Froissart) poles are bus-specific. The estimator takes the smallest pole magnitude agreed on by at least half of the sampled buses (clustered within `param.pole_cl_tol`), and falls back to the global smallest if no consensus forms. This replaces the legacy single-/random-bus pole pick and is one of the robustness improvements in this build (Section 6.5). To change it, edit `param.pole_buses`, `param.pole_consensus`, or `param.pole_cl_tol` in `solver_params.m`.

5.6 Programmatic single-case runs: `run_merged_case`

While the `run_batch*` drivers loop over a list of cases and write CSV summaries, `run_merged_case` is the **unified single-case harness**: it runs *one* case in *any* of the three frameworks behind a single function call and returns a structured, machine-readable result instead of printing to the console.

```
[result, solutions] = run_merged_case(case_name, mode, close_pool_on_finish)
%   mode = 'serial' | 'parfor' | 'parfeval'
```

It performs the identical preprocessing as the batch drivers (it reuses `main.m` as the setup template, substituting the requested case for the active `mpc=case...` line), traces the case in the selected framework, and **self-verifies** by recomputing the maximum residual $|z^\top M_i z - b_i|$ over every solution z and every equation i. The call never throws: on failure it returns a `FAIL` result rather than erroring. The returned `result` struct contains:

Field	Meaning
`status`	`'PASS'` or `'FAIL'`
`case_name`, `mode`	echoed inputs
`buses`	number of buses
`solutions`	number of solutions found
`max_residual`	largest constraint residual over the solution set (correctness check)
`wall_time`	trace wall-clock time (excludes pool start-up)
`error_id`, `message`	populated only on `FAIL`

The second output, `solutions`, is the full solution matrix `Zsave` (each column a solution). Typical uses: A/B comparison of the three frameworks on one case (identical setup, compare

`wall_time/solutions/max_residual`); automated regression or validation checks; and obtaining the solution set and residual directly as return values. Because it reads `main.m`, that file must remain in the folder for this harness to work.

5.7 Resuming a long run from a checkpoint

A large all-solutions run can take hours, so the collection scripts support resuming from a saved workspace instead of restarting. The parallel runbooks write periodic checkpoints to `temp_result.mat`. The saved state contains the model matrices, the current solution matrix `Zsave`, and its bookkeeping matrix `VBook`; runtime-only futures, pools, and optional `parallel.pool.Constant` objects are rebuilt on resume.

To resume a ceased `parfeval` or `parfor` search, return to the same solver folder, use the same case and load setting, load `temp_result.mat`, and run the same collection script:

```
load temp_result.mat
wait = 0;
run('runVBook_hybrid_parfeval.m')          % queue parfeval
% run('runVBook_hybrid_parallel.m')        % parfor
% run('runVBook_hybrid_parfeval_barrier.m') % retained barrier scheduler
```

The serial driver does not maintain a periodic checkpoint by default. Before stopping a serial run, save the workspace manually with `save('mycheckpoint.mat','-v7.3')`; resume with `load mycheckpoint.mat`, `wait = 0`, and `run('runVBook_hybrid_2023.m')`.

On start-up the script checks for an existing `VBook`. If `Zsave` is longer than `VBook`, it trims `Zsave` to the book-kept solution count; if their sizes match, it sets `initbypass` and continues without re-seeding. The queue `parfeval` driver rebuilds its pending work lists from the zero entries of `VBook`; all drivers re-seed the keyed deduplicator (Section 6.7) from the loaded solutions, so a large checkpoint continues at full speed.

6 How the solver works

6.1 Elliptical formulation

In rectangular voltage coordinates $x = [V_d^\top \; V_q^\top]^\top$, each power-flow equation is a quadratic $x^\top C_i x = b_i$. These native surfaces are unbounded (hyperbolic), so raw branch tracing can run off to infinity. Following Lesieutre–Wu (2015) and Wu's thesis, a positive-definite *base ellipsoid* is added to every equation, producing an equivalent system

$$x^\top M_i x = 1, \qquad M_i = M_i^\top \succ 0, \quad i = 1, \ldots, 2N_{\text{bus}},$$

whose zero set is *identical* to the original (algebraic-set preserving) but whose every equation is now a high-dimensional ellipse centred at the origin. Relaxing one equation with a tracing parameter α (the code variable `aal`) yields a 1-dimensional curve; because the ellipses are compact, each connected component of that curve is a **bounded loop** (generically diffeomorphic to a circle), so a trace must return to its start. Solutions occur wherever $\alpha = 0$ along the loop.

6.2 Holomorphic embedding and Padé

Rather than crawl the loop with small predictor–corrector steps, HEBC embeds the tracing parameter into the complex plane and represents each bus voltage as a holomorphic function, computed as a power series (to degree `degree.act`, default 15). All series degrees share a single

LU factorisation, so the coefficients are cheap. A **Padé approximant** (balanced numerator/-denominator degree, for maximal convergence radius) then extrapolates the voltages far along the curve in one step, until the power mismatch exceeds `param.mismatch_error`. The real roots of the Padé denominator reveal the location of the nearest singularity on the curve.

6.3 Hybrid switching and warm start

When a holomorphic step approaches a singularity — signalled either by the corrector failing to converge, or by the holomorphic increment $|\Delta\alpha|$ collapsing — the solver switches to predictor–corrector continuation to pass through the fold. To avoid a slow "cold" warm-up, a **warm starter** sets the first continuation step to a fraction of the estimated distance to the singularity (and no larger than a fraction of the last holomorphic step), then builds a quadratic predictor. After the fold is cleared, control returns to the holomorphic routine. One holomorphic step costs about 4× a continuation step but typically covers ≈ 8–9 continuation steps of arc, which is where the speed-up comes from.

6.4 Continuation details

The continuation phase uses a pseudo-arclength predictor (quadratic, from the last three points), a Newton corrector (*Phase I*), and a variable-rescaling corrector (*Phase II*) for badly-conditioned near-singular arcs whose secant slope exceeds `param.slope_max`. The step length adapts to the corrector iteration count toward a target of `param.targetsteps`. Each α sign change triggers a solution extraction and a duplicate check (Section 6.7); a trace terminates when it returns to its starting point (within `closure` tolerance) or exhausts the alternation cap `param.outnum`.

6.5 Validated, robustness-hardened configuration

This v4 build ships a default configuration that hardens the tracer against the numerical failures (wrong-branch drift and non-closing loops near sharp folds) that can occur with the legacy settings. Four changes matter most, all in `solver_params.m`:

- **Consensus pole estimate** (`pole_buses = 2:6`, `pole_consensus`) — rejects spurious per-bus poles that would place the hand-off badly (Section 5.5);
- **Turning-point slope limit** `slope_max = 4e5` — keeps the continuation active through steep near-singular arcs instead of handing back too early; validate difficult case118 curves before treating this threshold as a universal default;
- **Hand-back hysteresis** `handback = 0.5` — travels far enough past a turning point before returning to the holomorphic step that the curve cannot be misled onto an adjacent wrong branch;
- **Fold-proximity step caps** `init_cap_k = 1500` (caps the first continuation step after a hand-back) and `max_adapt = true` (an adaptive per-step cap driven by the secant-slope change) — both prevent a large step from jumping a sharp apex onto the wrong branch.

Unless you are experimenting, leave these defaults as shipped; the tables in Section 7 describe each setting.

6.6 Linear-solver acceleration: KLU symbolic-once refactor

The numerical hot path is the **bordered Newton solve** inside the Phase I corrector and inside the solution-resolving routine `Resolve_with_mex`. Each is a linear system whose $N_m \times N_m$ core is the power-flow Jacobian, bordered by one or two dense rows and columns (the arclength and scaling equations). This solve accounts for roughly 70–98% of the tracing time on large systems. The core, although stored densely by the classic code, is structurally *sparse*: only about 3% of its entries are nonzero, reflecting the network connectivity.

The solve is accelerated in three stackable steps, all of which keep the *exact* Jacobian — so quadratic Newton convergence and the exact solution set are unchanged:

1. **Sparse core.** `MakeJacobianD` returns the Jacobian core directly as a sparse matrix, avoiding a dense factorisation and any per-iteration `sparse()` conversion.

2. **Schur complement.** The one or two dense border rows and columns are eliminated against the sparse core, leaving only a 1–2 dimensional dense system to finish. Concretely, write the bordered Jacobian and right-hand side in block form, with the sparse power-flow core $A = J \in \mathbb{R}^{N_m \times N_m}$ in the top-left and m thin dense border rows and columns ($m = 2$ in the continuation corrector — the arclength and scaling equations — and $m = 1$ in `Resolve_with_mex`):

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix}, \qquad A \in \mathbb{R}^{N_m \times N_m},\ \ B \in \mathbb{R}^{N_m \times m},\ \ C \in \mathbb{R}^{m \times N_m},\ \ D \in \mathbb{R}^{m \times m}.$$

   Only A is large and sparse. Eliminating the block $y_1 = A^{-1}(f_1 - B\,y_2)$ leaves the small $m \times m$ *Schur-complement* system for the border unknowns,

$$S\,y_2 = f_2 - C\,g, \qquad S = D - C\,H, \qquad \text{with} \quad A\,g = f_1,\ \ A\,H = B,$$

   after which $y_1 = g - H\,y_2$ and $dy = -[\,y_1;\ y_2\,]$. The two core solves $g = A^{-1}f_1$ and $H = A^{-1}B$ share a *single* factorisation of A and are obtained in one call, $\texttt{GH} = \texttt{klurf}(A, [\,f_1 \ \ B\,])$, with g the first column and H the remaining m; so the whole bordered solve costs one sparse factorisation of A (a cheap `klu_refactor` after the first iteration) plus one $m \times m$ dense solve, and the dense $(N_m{+}m) \times (N_m{+}m)$ matrix J_1 is never assembled.

3. **KLU symbolic-once refactor.** The sparse core is factorised with **KLU** (SuiteSparse; T. Davis). Because the sparsity pattern is identical across every Newton iteration and every curve of a case, the fill-reducing ordering and symbolic analysis are computed *once* (`klu_analyze`); each later solve only recomputes numeric values on that ordering (`klu_refactor`), skipping the ordering, symbolic analysis, and pivot search that a fresh factorisation repeats every time.

The speed-up grows with the system size $N_m = 2N_{\text{bus}} - 1$ (measured serially on an Intel i9-13900K, MATLAB R2022a; solution counts identical to the dense solve on every case):

System	N_m	full-factor KLU	symbolic-once refactor
$\leq$ case14mod	≤ 27	$\approx$1.0×	$\approx$1.0× (overhead-bound)
case30 / case33bw	59 / 65	$\approx$1.0×	1.1–1.25×
case39	77	$\approx$1.0×	1.43×
case118	235	2.35×	**7.5**×

Below $N_m \approx 55$ the matrices are small enough that fixed overhead dominates and the accelerated path is a wash (occasionally a few percent slower); the gain grows monotonically with N_m and is largest exactly where run time matters. No size gate is applied.

Enabling and overriding. The refactor path is provided by the `klurf` MEX, built from SuiteSparse/KLU. This build ships `klurf.mexw64` (Windows 64-bit), so acceleration is **on by default**; if the MEX is absent the solver transparently falls back to the dense solve. To force a mode for benchmarking, set the global before running:

`global USEKLU; USEKLU = 0;`	dense bordered solve
`USEKLU = 1;`	KLU full factorisation each iteration (needs `klu.mexw64`)
`USEKLU = 2;`	KLU symbolic-once refactor (`klurf`) — default when present
`USEKLU = [];`	auto: refactor if `klurf` present, else dense (the default)

Correctness is validated by reproducing the exact dense-solve solution counts on every system up to case39 — including the ill-conditioned case39 (turning-point slopes $\sim 10^9$), where a reused pivot ordering is most at risk.

6.7 Solution deduplication: keyed bucket lookup

As each trace returns candidate solutions, every candidate is checked against the solutions already found — different traces routinely re-find the same roots, and only genuinely new ones are recorded. The classic code did this with a linear scan: each candidate was compared (L_1 distance within `4e-7`, exact and antipodal) against the *entire* growing solution set, which is $O(N)$ per candidate and $O(N^2)$ over a run. On a large resumed run this dominates: at $N \approx 2.3 \times 10^6$ solutions the scan costs $\sim$1 s per candidate, so resuming a big checkpoint crawls while a fresh start (small N) runs fast.

`KeyedDedup.m` replaces the scan with a hash-bucketed lookup. A 1-Lipschitz key — the sum over a few fixed, seeded-random *signature* variables — buckets the solutions, so a candidate is compared only against its own bucket and the two neighbouring buckets: $O(1)$ amortised instead of $O(N)$. The match is the same tolerance test on those signature variables (exact and antipodal). Seeded once from the current solution set, one object serves the serial, parfor, and parfeval collectors alike. Solution counts are unchanged — validated on `case9`/`case14mod` and on a 2.3×10^6-solution checkpoint — while the per-candidate cost drops from $\sim$1 s to $\sim 40\,\mu$s ($\sim 10^4\times$), which is what makes resuming a large run practical.

7 Parameter reference

All tunables live in `solver_params.m` — a single function returning the `param` struct that every framework and every parallel worker reads. Edit a line there and the change applies everywhere. The file is fully commented; the tables below cover the parameters you are most likely to adjust, with this build's shipped defaults.

7.1 Continuation / step control

Parameter	Default	Meaning
`sstep`	`1e-5`	Base arc-length step scale for the continuation.
`targetsteps`	`3`	Target number of corrector iterations; drives step-size adaptation.
`imax`	`40`	Maximum Newton iterations per corrector step.
`smax`	`20000`	Maximum continuation steps per `branch_trace` call.
`maximumstep`	`8000`	Maximum step-size multiplier (Phase I ceiling base).

Parameter	Default	Meaning
maxphase1	maximumstep/2	Maximum step in Phase I.
maxphase2	maxphase1/10	Maximum step in Phase II.
minimumstep	1e-2	Minimum step-size multiplier.
ef	1e-8	Corrector residual tolerance $\|F\|$.
ex	1e-7	Corrector step tolerance $\|\Delta x\|$.
PS	300	Phase II (rescaling) duration, in steps.
maxsolu	100	Maximum solutions collected per traced loop.
maxskip	50	Consecutive numerical stalls tolerated before the loop is skipped.
dev	20	Console print interval (display only).

7.2 Holomorphic / hand-off control

Parameter	Default	Meaning
mismatch_error	1e-4	Maximum holomorphic power-mismatch accepted before shrinking the arc.
holonum	50	Maximum Padé steps per outer iteration before forcing continuation.
holo_arc_max	0.01	Maximum holomorphic arc length per step.
outnum	500	Cap on holomorphic↔continuation alternations per curve (a stall burns this cap).
slope_max	4e5	Maximum secant slope at which a turning point is accepted as passed. Validate difficult case118 curves before generalizing this threshold.
aal_min	1e-6	Minimum homotopy-parameter increment; the unit for handback.
im_tol	1e-3	Imaginary-part tolerance for accepting a Padé root as a real pole.
handback	0.5	Hand-back hysteresis (×aal_min): how far past a turning point to travel before returning to the holomorphic routine. 0.5 is validated across all cases (the legacy build used 0.1).
interval	1	Stride for the turning-point finite-difference test.

7.3 Consensus pole estimation

Parameter	Default	Meaning
pole_buses	2:6	Buses sampled for the consensus pole estimate (auto-filtered to $\leq$ bus count).
pole_consensus	true	If true, take the smallest pole agreed on by $\geq$ half the sampled buses; else the global minimum.
pole_cl_tol	0.10	Relative magnitude tolerance for clustering "the same pole" across buses.

7.4 Fold-proximity and stall control

These are the robustness controls added in v3 (Section 6.5).

Parameter	Default	Meaning
`init_cap_k`	`1500`	Caps the first continuation step after a hand-back at `init_cap_k`×`holo_arc`/`sstep`, so a fold entry cannot be overshot (`Inf` disables).
`max_adapt`	`true`	Adaptive per-step ceiling: `astep` $\leq$ `maximumstep` $\times \min(s_{k-1}/s_k, 1)$ from the 2-point secant slope s — shrinks the step approaching a fold, restores it after. `false` disables.
`aal_limit_init`	`30`	Sentinel magnitude meaning "no Padé pole found yet".
`stall_iter_mult`	`3`	Flag a numerical stall when corrector iterations exceed this ×`targetsteps`.
`step_adapt_denom`	`40`	Step-growth denominator: `astep` ×(1 + (targetsteps − iters)/this).

7.5 Holomorphic degree

Set in the drivers / `main.m`: `degree.act = 15` (active power-series degree), split as `degree.numerator = 8`, `degree.denominator = 7` for the Padé approximant. **The shipped MEX bake these degree values in as compile-time constants** — if you change them, regenerate `examples.mat` and rebuild the MEX (Section 11).

8 Release benchmark comparison

The 2026.07.15 release benchmark ran the queue `parfeval` driver at nominal load on all 20 bundled cases through 57 buses using MATLAB R2022a on `PCWIN64` with 23 local workers. Parallel-pool startup was excluded from the per-case timings. MEX v4 completed the suite in 718.656 s (716.062 s trace time), while the pure-MATLAB v4 companion required 1272.966 s (1271.222 s trace time): 1.77× less aggregate wall time for this MEX package. MEX was faster on 14 cases; the six small-case reversals are overhead-sensitive.

Both packages returned the same solution count on all 20 cases (3256 total). An order-independent one-to-one solution-set comparison had maximum nearest-solution distance 3.154×10^{-7}, below the shared `4e-7` deduplication tolerance. This benchmark compares the two 2026.07.15 v4 packages only, not earlier v2/v3 or 2026.07.14 releases. The complete per-case table and methodology are in the top-level `HEBCPF_Suite_Overview.pdf`; the CSV and report are under `benchmark_results_20260715/20260715_194046/`.

9 Outputs

Console: per-case progress, solution count, and CPU (serial) or wall (parallel) time, followed by a summary table.

CSV summaries (written into the run folder):

Driver	CSV file
`run_batch.m`	`results_summary.csv`
`run_batch_par.m`	`results_par_summary.csv`
`run_batch_parfeval.m`	`results_parfeval_summary.csv`

Columns: `case_name`, `cpu_time_sec` (or wall time), `n_solutions`, `status` (OK / ERROR), `error_msg`. These CSV files are run outputs and are ignored by the repository `.gitignore`.

The full solution set is available in the workspace as `Zsave` (each column a solution in the internal variable ordering) after a run; the solver sign-normalises and deterministically orders it (`canonicalize_solutions.m`) so that serial, parfor and parfeval produce identical output for the same case. Antipodal pairs $\{x, -x\}$ are identified as one solution.

10 File map

Configuration: `solver_params.m` — single source of truth for every tunable constant (Section 7).

Setup / preprocessing: `makeYbus` from MATPOWER (Ybus), `get_quadr_mtrx.m` (power-balance quadratics), `quadr_matrix.m` (per-bus PV/PQ/slack matrices), `Solu.m` (initial point from `runpf`), `MakeEllipse.m` (base-ellipse generation), `Tune_Fac.m` (continuation scaling), `main.m` (reference preprocessing script; its logic is inlined into the drivers).

Core solver (shared hot path): `hybrid_traceloops_4_with_mex.m` (outer trace-loop orchestrator; loads `param` from `solver_params.m`, primes the cached operators), `holomorphic_hybrid_5_with_mex.m` (one holomorphic step + consensus pole estimate), `branch_trace_hybrid_4_with_mex.m` (predictor–corrector continuation), `Resolve_with_-mex.m` (Newton corrector), `holomorphic_cont_tri.m`(+MEX) (Taylor-coefficient triangular solves), `holomorphic_para_sp.m` (sparse H builder), `Pade_Apprxmt.m`(+MEX) (Padé construction), `update_V_Pade.m` (Padé evaluation along the arc), `MakeJacobianD.m` (cached sparse Jacobian), `quad_form_vals.m` (cached constraint-residual evaluator), `MakeJacobian.m` (scalar Jacobian, used by `Tune_Fac`).

Post-processing: `canonicalize_solutions.m` (sign-normalise + deterministic ordering).

Deduplication: `KeyedDedup.m` (bucketed $O(1)$ duplicate lookup shared by all three collectors; see Section 6.7).

Parallel housekeeping: `cleanup_parallel_artifacts.m` (clears per-worker caches, optionally closes the pool to avoid a Windows/R2022a `-batch` shutdown crash).

Drivers: serial — `runVBook_hybrid_2023.m`, `run_batch.m`; parfor — `runVBook_hybrid_-parallel.m`, `trace_equation_worker.m`, `run_batch_par.m`; parfeval — `runVBook_hybrid_-parfeval.m`, `trace_equation_worker_const.m`, `trace_equation_worker.m`, `run_batch_-parfeval.m`; convenience — `run_merged_case.m` (one case, any mode).

Build: `build_mex.m`, `examples.mat` (codegen type seeds).

Case data: `caseXXX.m` MATPOWER-format test systems.

11 Rebuilding the MEX

The shipped binaries are Windows 64-bit. On other platforms, or after editing any hot-path source, rebuild with MATLAB Coder:

```
>> cd '<path>\HEBCPF_MEX_v4_20260715'
```

```
>> addpath(pwd)
>> build_mex     % rebuilds both .mexw64 from the .m source + examples.mat
```

`build_mex.m` defines the codegen argument types (`degree` as `coder.Constant`; the `I.pv`/`I.pq` index fields typed variable-size because `case3` has no PV bus). If you change `degree.act`, `numerator` or `denominator`, regenerate `examples.mat` (save the `degree`, `I` and `Start` structs from any quick run) and rebuild, because the MEX are specialised to those constant degree values.

Windows build gotcha. If codegen fails with "`Pade_Apprxmt_mex.bat is not recognized`", the environment variable `NoDefaultCurrentDirectoryInExePath` is set, blocking codegen's generated `.bat`. Unset it *before* launching MATLAB:

```
bash :  unset NoDefaultCurrentDirectoryInExePath ; matlab ...
cmd  :  set "NoDefaultCurrentDirectoryInExePath=" && matlab ...
```

Also try `mex -setup C` and re-selecting the compiler (developed with MSVC 2019). If you have no compiler, use the pure-MATLAB build. It matched the released solution sets; its measured queue-`parfeval` performance is reported in Section 8.

To run *without* the MEX (A/B comparison), comment the `_mex` call lines in `holomorphic_hybrid_5_with_mex.m` and uncomment the corresponding `.m` calls.

12 Troubleshooting

`Undefined function 'runpf'`
MATPOWER is not on the MATLAB path. Install and `addpath` it.

`Undefined function 'makeYbus'`
MATPOWER is not on the MATLAB path, or the MATPOWER `lib` folder is missing from the path.

`Undefined function 'caseXXX'`
Run `addpath(pwd)` from this folder; the case data files live here.

`Undefined function 'Pade_Apprxmt_mex'` / "generated for a different platform"
You are not on Windows x64, or the binary moved. Run `build_mex`, or use a pure-MATLAB build.

`codegen ".bat is not recognized"`
See the Windows build gotcha (Section 11).

OSQP warning during `runpf`
Harmless MATPOWER solver-probe message; `runpf` still converges (equation error ~ 0).

Parallel driver appears to hang at first run
The local `parpool` is starting; this one-time cost is excluded from the reported timing.

Solution count differs from the expected value
Check the load factor is 1 and that the MATPOWER initial solve (`runpf`) reported success. Note that the pole-estimation, `handback` and `slope_max` settings affect *tracing efficiency and numerical robustness*, not the final solution set on well-behaved cases.

13 License and third-party notices

HEBCPF is distributed under the BSD 3-Clause License; see the top-level `LICENSE` file. Some bundled `caseXXX.m` files are copied from, derived from, or modified from MATPOWER and other public test-system sources. Preserve the per-file attribution notices when redistributing modified versions, and see the top-level `NOTICE` file for third-party attribution details.

The optional KLU acceleration (Section 6.6) links **SuiteSparse/KLU** (Timothy A. Davis), distributed under the GNU LGPL-2.1-or-later license. The `klurf` MEX source (`klurf/klurf_mex.c`) and its build script are included; see `klurf/README_KLURF.md` and the SuiteSparse license terms for details.

14 Citing this solver

If you use results produced by this solver, please cite the HEBC paper:

> D. Wu and B. Wang, "Holomorphic Embedding Based Continuation Method for Identifying Multiple Power Flow Solutions," *IEEE Access*, vol. 7, pp. 86843–86853, 2019. doi: 10.1109/ACCESS.2019.2925384.

BibTeX:

```
@article{wu2019hebc,
  author  = {Wu, Dan and Wang, Bin},
  title   = {Holomorphic Embedding Based Continuation Method for
             Identifying Multiple Power Flow Solutions},
  journal = {IEEE Access},
  volume  = {7},
  pages   = {86843--86853},
  year    = {2019},
  doi     = {10.1109/ACCESS.2019.2925384}
}
```

For citing the software package itself, see the top-level `CITATION.cff` file.

Further reading

The method rests on, and is extended by, the following works:

1. B. C. Lesieutre and D. Wu, "An Efficient Method to Locate All the Load Flow Solutions — Revisited," in *2015 53rd Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, Monticello, IL, USA, pp. 381–388, IEEE, 2015. doi: 10.1109/ALLERTON.2015.7447029. *(The elliptical branch-tracing formulation that underlies this solver.)*

2. D. Wu, "Algebraic Set Preserving Mappings for Electric Power Grid Models and Its Applications," Ph.D. dissertation, University of Wisconsin–Madison, 2017. *(The dissertation that formalises the elliptical / algebraic-set-preserving formulation and specifies the predictor–corrector continuation machinery — the Phase I/II correctors, step-length control, and solution/termination logic — that this solver implements.)*

3. D. Wu, D. K. Molzahn, B. C. Lesieutre, and K. Dvijotham, "A Deterministic Method to Identify Multiple Local Extrema for the AC Optimal Power Flow Problem," *IEEE Transactions on Power Systems*, vol. 33, no. 1, pp. 654–668, Jan. 2018. doi: 10.1109/TPWRS.2017.2707925. *(Extends the tracing method to AC optimal power flow.)*

4. B. Wang, D. Wu, X. Su, K. Sun, and L. Xie, "A Long-Term Voltage Stability Margin Index Based on Multiple Real Power Flow Solutions," in *2023 IEEE Power & Energy Society General Meeting (PESGM)*, Orlando, FL, USA, pp. 1–5, IEEE, Jul. 2023. doi: 10.1109/PESGM52003.2023.10252776. *(Defines a voltage-stability margin as the minimum voltage-space distance from the high-voltage solution to all other real power-flow solutions — a direct application of this solver's output.)*

5. D. Wu and B. Wang, "Influence of Load Models on Equilibria, Stability and Algebraic Manifolds of Power System Differential-Algebraic System," in *2019 57th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, Monticello, IL, USA, IEEE, 2019. doi: 10.1109/ALLERTON.2019.8919649. *(How load models reshape the solution manifold and its equilibria.)*

6. D. Wu and B. Wang, "Collection of Numerous Power Flow Solutions of Standard IEEE Test Systems," *IEEE Dataport*, Jul. 2019. doi: 10.21227/24bh-hj72. *(The published dataset of reference solution sets for the standard IEEE test systems — useful for validating this solver's output.)*